

Optimizing High Performance Data Processing for Large-Scale Web Crawlers

Project 1

Name: Ahmad Ziyaad Bin Mohd Abbas

Matric: A23CS0206

Lecturer: Dr Mohd Shahizan Bin Othman

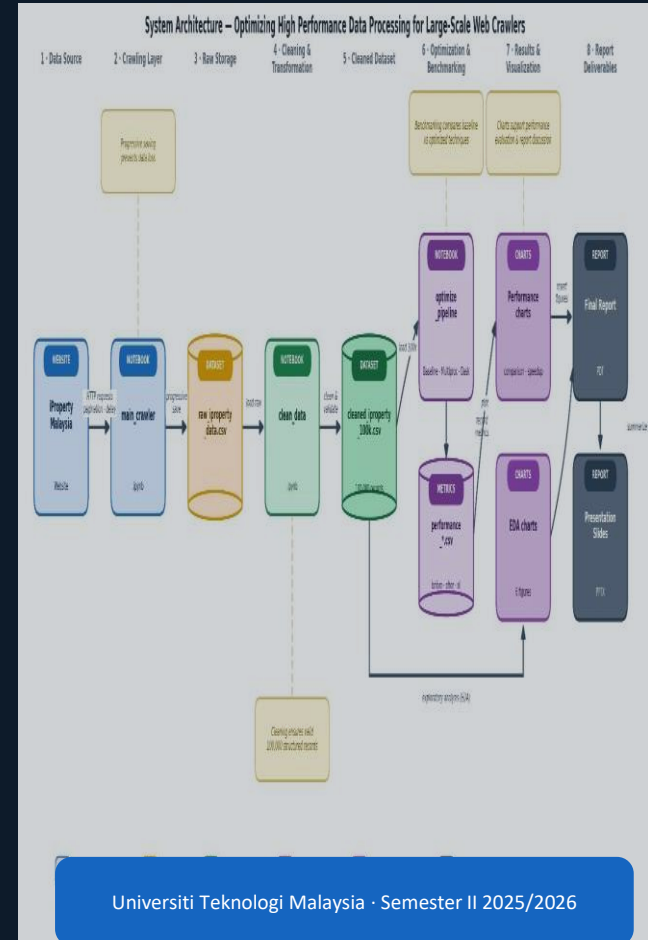


Table of Contents

01

Introduction

Background, objectives & scope

02

Target Website & Dataset

iProperty Malaysia, 100K records, 17 fields

03

System Architecture

Three-stage modular pipeline

04

Web Crawling Methodology

Ethical scraping, progressive saving

05

Data Cleaning

Deduplication, standardization, 100K valid records

06

Exploratory Data Analysis

6 EDA charts — region, price, category

07

Optimization Techniques

Baseline, Multiprocessing, Dask

08

Performance Evaluation

Benchmarking results & analysis

09

Challenges & Conclusion

Findings, limitations, future work

Background & Objectives

- 01 Collect 100K+ Records**
Scrape all Malaysian states from iProperty Malaysia, exceeding the 100,000-record project threshold.
- 02 Structure & Clean Data**
Store in CSV format with deduplication, field standardization, and type validation.
- 03 Implement HPC Techniques**
Apply Multiprocessing (ProcessPoolExecutor) and Dask as two distinct optimization strategies.
- 04 Benchmark & Evaluate**
Measure execution time, CPU%, peak memory, and throughput across all three approaches.
- 05 Interpret Results Honestly**
Analyse when parallelization helps vs. hurts — the key engineering lesson of the project.

iProperty Malaysia — Dataset Description

100,004

Raw Records Collected

100,000

Valid Records (Cleaned)

64,173

For-Sale Listings

35,827

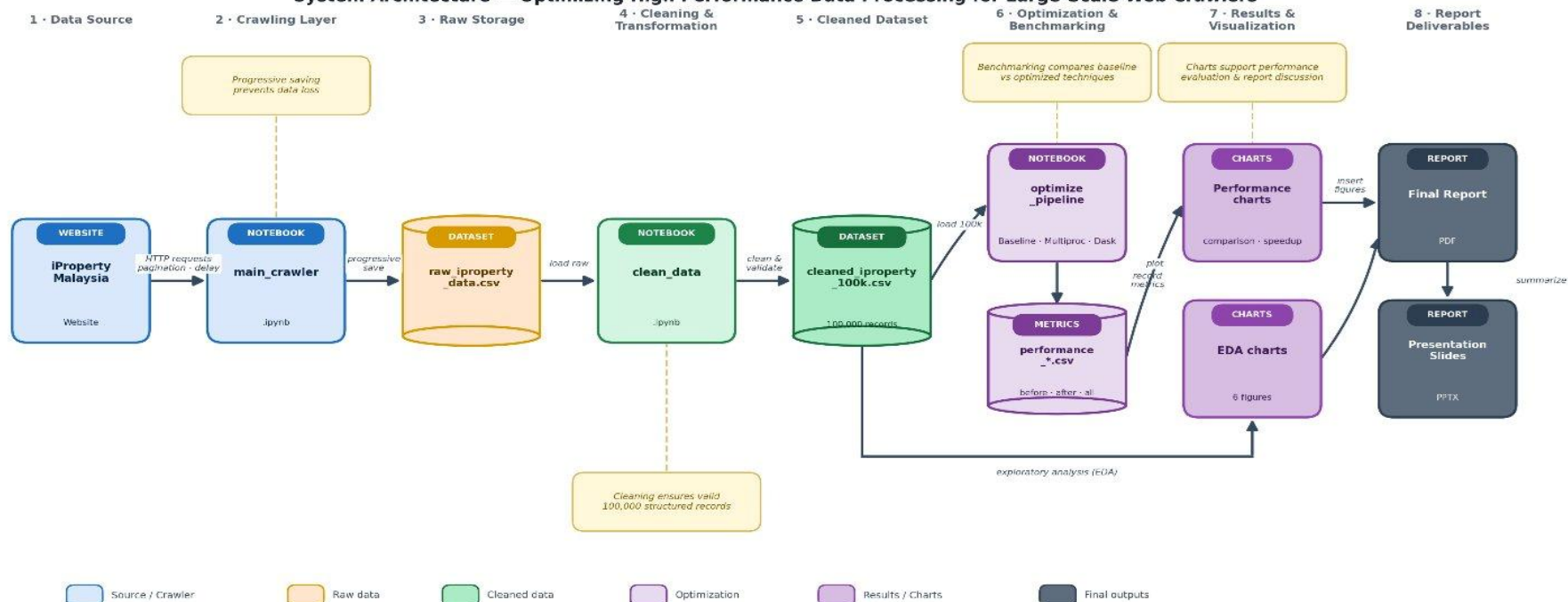
For-Rent Listings

17 Extracted Fields (key subset shown)

Field	Description
listing_id	Unique listing identifier
listing_type	for-sale or for-rent
price	Listed price (RM)
location / region	Sub-locality and state-level region
bedrooms / bathrooms / car_parks	Room and parking counts
size_sqft	Built-up / land area in sq. ft.
property_type / category	Raw and standardized classification
tenure / furnishing	Freehold/leasehold & furnishing status

System Architecture — Three-Stage Pipeline

System Architecture — Optimizing High Performance Data Processing for Large-Scale Web Crawlers



Data flows from web source → progressive CSV storage → cleaning/transformation → HPC benchmarking → final reports

Crawling Methodology & Ethical Considerations

1 Seed & Enumerate

Seed listing pages by region (all states) and listing type (for-sale / for-rent), then enumerate paginated result sets.

2 Parse & Extract

For each page, parse listing cards and extract the 17 structured attributes including price, location, and property type.

3 Progressive Save

Append records to CSV incrementally rather than buffering in memory — prevents data loss on network or parsing failures.

4 Crawl Delay & Retry

Apply deliberate crawl delays between requests and retry-with-backoff on transient HTTP failures.

5 100K Target

Continue until at least 100,000 valid records are collected; duplicate URLs are discarded during collection.

Ethical Guidelines

- robots.txt reviewed & honored
- Deliberate crawl delay applied
- Non-deceptive User-Agent used
- Only public attributes collected
- No auth walls bypassed
- Academic use only, not redistributed

Cleaning Pipeline & Missing Values

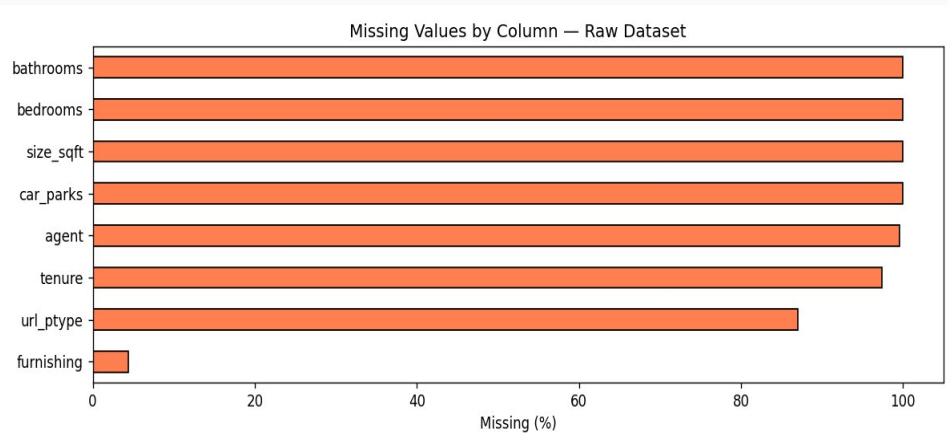


Figure 7.1 — Missing Values by Column (Raw Dataset)

1

Duplicate Removal

Exact & key-based duplicate listings dropped

2

Field Standardization

price_rm parsed, listing_type / region / furnishing normalized

3

Property Categorization

Free-text → Terrace House, Apartment, etc. (Unknown if unmatched)

4

Type Validation

Numeric fields coerced; invalid values nulled (not dropped)

5

Final Selection

17 well-defined columns, exactly 100,000 valid records

EDA — Listings, Regions & Price

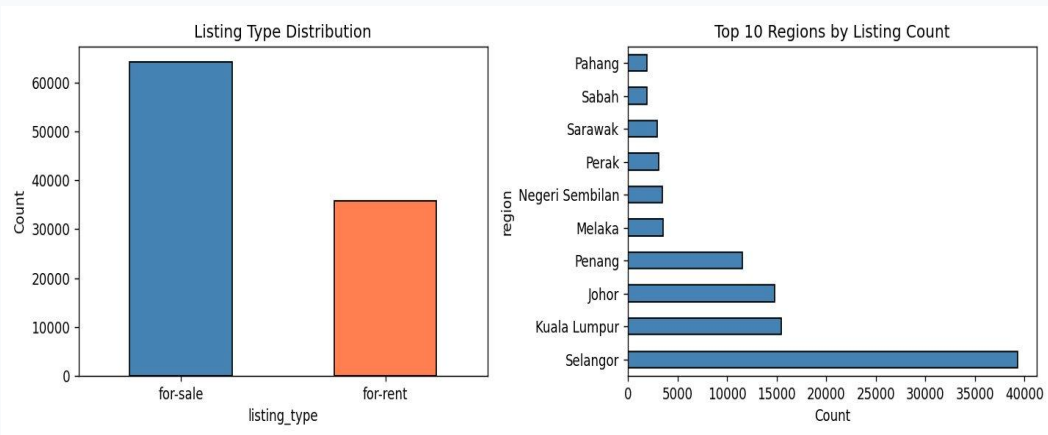


Fig 8.1 — Listing Type & Top Regions by Count

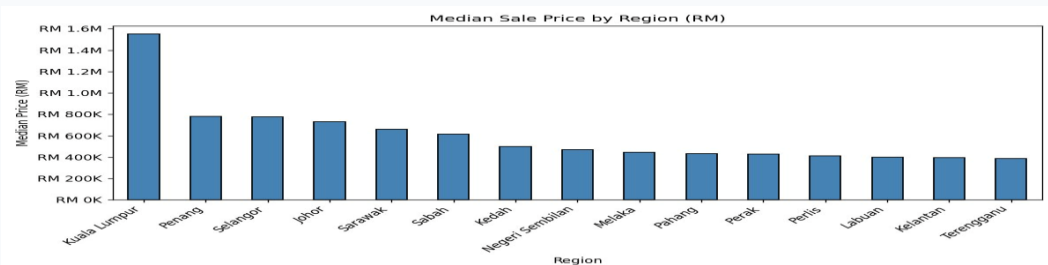


Fig 8.2 — Median Sale Price by Region (RM)

64,173 vs 35,827

For-sale listings outnumber for-rent in a roughly 64:36 ratio.

Selangor #1

Dominates listing volume, followed by Kuala Lumpur, Johor, and Penang.

KL Median RM 1.55M

Kuala Lumpur commands the highest median sale price — consistent with urban premium.

Penang & Selangor

Second and third highest median prices; reflect major economic centres.

EDA — Price Distribution, Furnishing & Categories

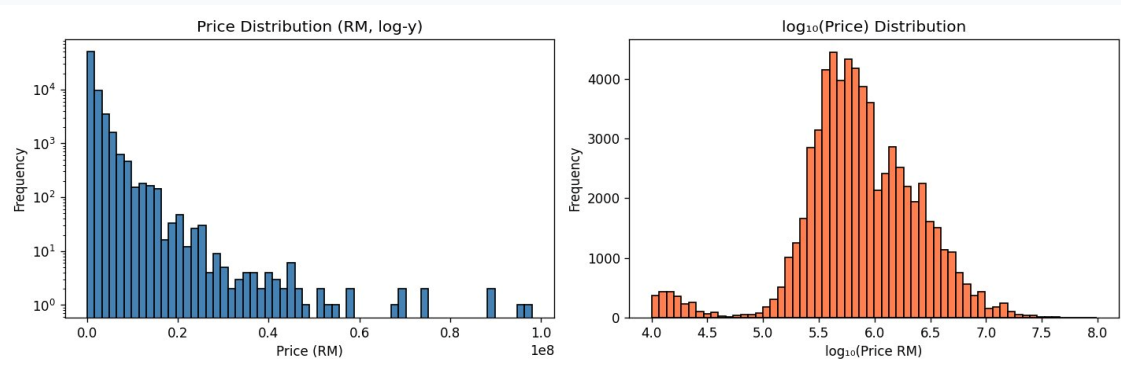


Fig 8.3 — Raw & Log-Transformed Price Distribution

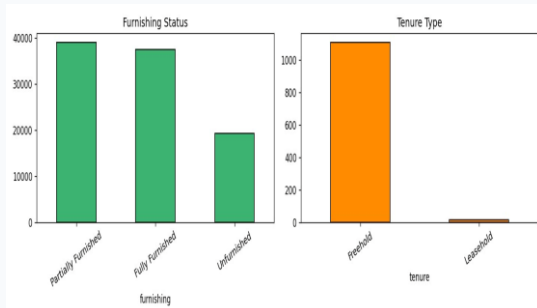


Fig 8.4 — Furnishing & Tenure

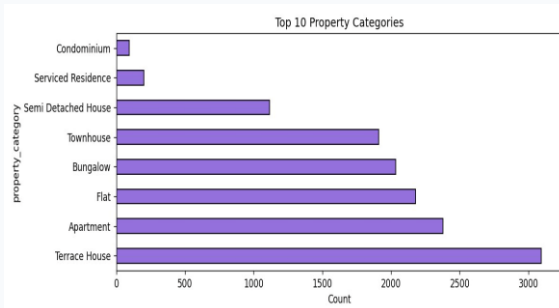


Fig 8.5 — Top Property Categories

Key Insights

- Right-skewed raw price; log₁₀ transform produces near-normal bell curve → better for modelling
- Partially furnished & fully furnished dominate; unfurnished is least common
- Among listings with tenure: Freehold vastly outnumbers Leasehold
- Terrace House is the most common category, followed by Apartment and Flat

Three Strategies, One Benchmark Pipeline

01**Baseline**

Sequential pandas

- Eager, single-threaded execution
- Full dataset loaded into RAM
- Vectorized pandas operations
- Reference point for speedup

02**Multiprocessing**

ProcessPoolExecutor (7 workers)

- Process-level parallelism
- Bypasses Python's GIL
- Workers = CPU cores - 1
- CPU-bound workload distribution

03**Dask**

dask.dataframe + LocalCluster

- 28 partitions (workers × 4)
- Lazy evaluation & task graph
- Distributed-style scheduling
- Designed for out-of-core data

5-Task Benchmark Pipeline

Data Ingestion → Feature Engineering → Data Standardisation → Type Casting → Aggregation | Each strategy run 3× on 100K rows

Benchmark Results — 100,000 Rows

Performance Comparison — Baseline vs Multiprocessing vs Dask
Dataset: iProperty Malaysia (100,000 rows)

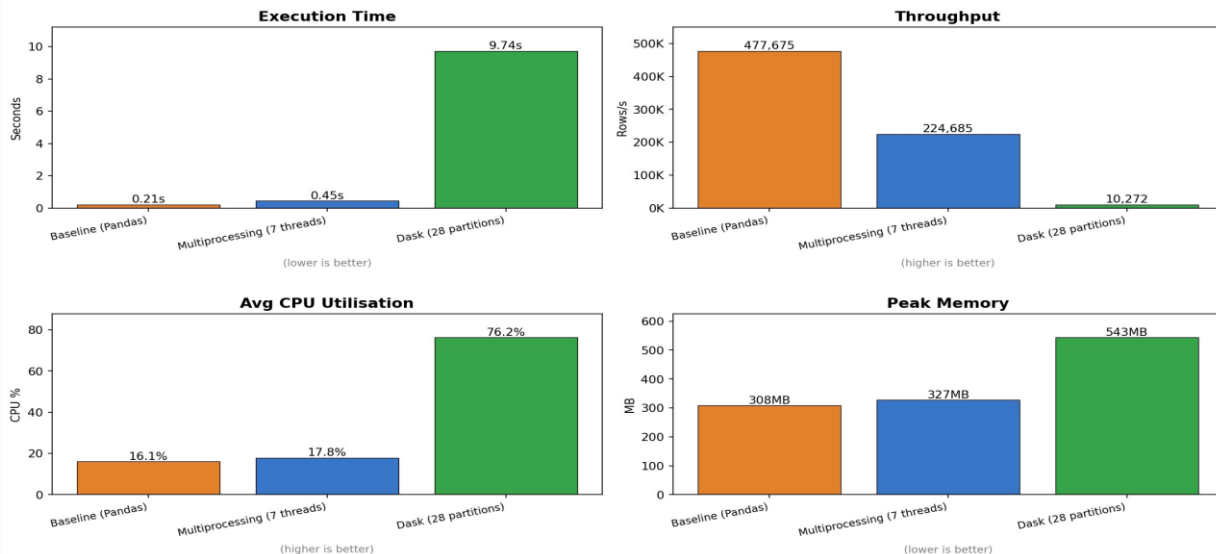


Fig 10.1 — Performance Comparison

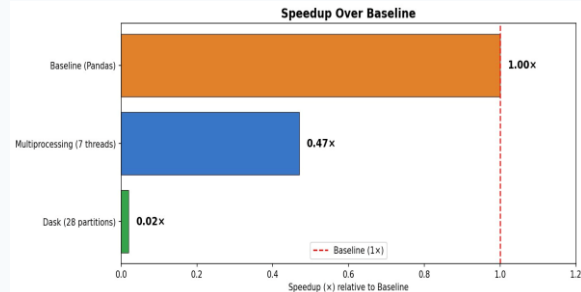


Fig 10.2 — Speedup Over Baseline

Metric	Baseline (pandas)	Multiprocessing	Dask (28 partitions)
Execution time (s)	0.21 ✓	0.45	9.74
Throughput (rows/s)	477,675 ✓	224,685	10,272
Avg CPU util. (%)	16.1	17.8	76.2
Peak memory (MB)	308 ✓	327	543
Speedup vs baseline	1.00x	0.47x	0.02x

Why Pandas Won — The Engineering Lesson

Parallelization Overhead > Compute Time

At 100K rows, pandas vectorized operations complete in 0.21 s. Spawning processes, IPC serialization, and Dask's task-graph coordination all exceed the actual compute saved.

Dask Scheduler Overhead

Dask added 9.74 s (46× slower) due to LocalCluster startup, 28-partition scheduling, and distributed communication — none of which Pandas pays.

CPU Utilization ≠ Speed

Dask drove 76.2% avg CPU (vs 16.1% baseline), confirming cores were engaged — but the extra activity coordinated rather than computed, hence no speedup.

Memory Follows Complexity

Dask peaked at 543 MB vs pandas' 308 MB. Partition management and scheduler state add overhead proportional to the number of partitions.

When Would Parallel Win?

At 5–50 M rows, or with heavy per-row transforms (geocoding, NLP, inference), compute dominates overhead and the ordering reverses. Always benchmark first.

Challenges, Limitations & Next Steps

Challenges & Limitations

Source Data Sparsity

bathrooms, bedrooms, tenure, agent mostly missing; `property_category` resolves for only ~13% of listings.

Small Benchmark Workload

At 100K rows the transform finishes in <1 s under pandas, leaving no headroom for parallel speedup.

Single-Machine Constraint

Dask ran on LocalCluster — true multi-node scaling advantages could not be realized.

Crawling Fragility

Pagination changes, rate limiting, and network failures required defensive error handling and progressive saving.

Future Work

Scale to 5–50M Rows

Find the crossover point where multiprocessing and Dask overtake the baseline.

Heavier Transforms

Introduce per-row geocoding, text NLP, or model inference to make workloads compute-bound.

Genuine Cluster Deploy

Run Dask on a true multi-node cluster to evaluate distributed scaling.

Alternative Engines

Evaluate Polars or DuckDB, which often outperform pandas without distributed overhead.

Persistent Storage

Migrate to PostgreSQL or MongoDB for scalable querying beyond flat CSV.

Key Takeaways

1

Pipeline Delivered

100,000 valid structured iProperty records collected, cleaned, and benchmarked end-to-end.

2

Honest Results

Sequential pandas baseline outperformed both parallel techniques at 100K rows — parallelization overhead dominated.

3

Core Engineering Lesson

Parallelization must be justified by workload size. Always benchmark before optimizing.

4

Future Path

Scaling to millions of rows or heavier transforms is where Multiprocessing and Dask will demonstrate their true value.